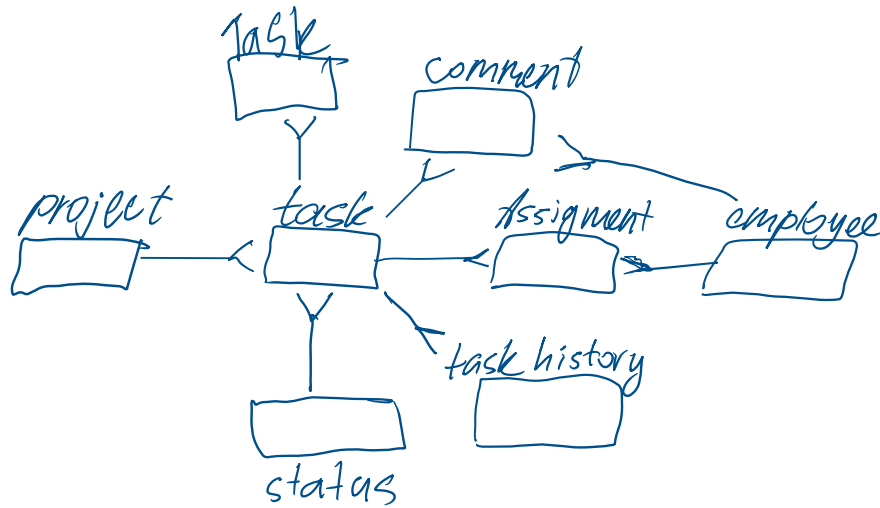




1. Project

- `project_id` (PK, INTEGER) - Идентификатор проекта.
- `name` (VARCHAR) - Название проекта.
- `description` (TEXT) - Описание.
- `start_date` (DATE) - Дата начала.
- `end_date` (DATE) - Плановая дата окончания.
- `manager_id` (FK -> Employee.employee_id) - Руководитель проекта.

2. Employee

- `employee_id` (PK, INTEGER) - Идентификатор сотрудника.
- `full_name` (VARCHAR) - Полное имя.
- `email` (VARCHAR) - Электронная почта.
- `position` (VARCHAR) - Должность.
- `department` (VARCHAR) - Отдел.

3. Status

- `status_id` (PK, INTEGER) - Идентификатор статуса.
- `status_name` (VARCHAR) - Название статуса (Новая, В работе, На проверке, Завершена).
- `is_closed` (BOOLEAN) - Признак завершающего статуса.

4. Task

- `task_id` (PK, INTEGER) - Идентификатор задачи.
- `project_id` (FK -> Project.project_id) - Проект.
- `parent_task_id` (FK -> Task.task_id, NULL) - Ссылка на родительскую задачу (для иерархии).
- `status_id` (FK -> Status.status_id) - Текущий статус.

- title (VARCHAR) - Заголовок задачи.
- description (TEXT) - Описание.
- priority (VARCHAR) - Приоритет (Высокий, Средний, Низкий).
- deadline (DATE) - Срок выполнения.
- created_by (FK -> Employee.employee_id) - Кто создал.
- created_at (DATE) - Когда создал.

5. Assignment

- assignment_id (PK, INTEGER) - Идентификатор назначения.
- task_id (FK -> Task.task_id) - Задача.
- employee_id (FK -> Employee.employee_id) - Исполнитель.
- assigned_date (DATE) - Дата назначения.

6. Comment

- comment_id (PK, INTEGER) - Идентификатор комментария.
- task_id (FK -> Task.task_id) - Задача.
- employee_id (FK -> Employee.employee_id) - Автор.
- comment_text (TEXT) - Текст комментария.
- created_at (TIMESTAMP) - Дата и время создания.

7. TaskHistory

- history_id (PK, INTEGER) - Идентификатор записи.
- task_id (FK -> Task.task_id) - Задача.
- changed_by (FK -> Employee.employee_id) - Кто изменил.
- old_status_id (FK -> Status.status_id) - Старый статус.
- new_status_id (FK -> Status.status_id) - Новый статус.
- changed_at (TIMESTAMP) - Дата изменения.
- changed_why (TEXT) - Причина изменения

Все связи — неидентифицирующие, так как у каждой дочерней сущности есть собственный РК

Задание 4:

1. Показать все задачи конкретного проекта с их текущими статусами и ФИО исполнителей
2. Найти все просроченные задачи со статусом, не равным «Завершена»

3. Вывести список всех сотрудников с количеством активных (незавершённых) задач, назначенных на них, отсортировав по убыванию
4. Показать историю изменения статуса для конкретной задачи: кто, когда, с какого на какой статус и почему
5. Рассчитать среднее время выполнения задач (в днях) по каждому проекту

Задание 3:

-- Удаляем таблицы, если они существуют

```
DROP TABLE IF EXISTS taskhistory CASCADE;
```

```
DROP TABLE IF EXISTS comment CASCADE;
```

```
DROP TABLE IF EXISTS assignment CASCADE;
```

```
DROP TABLE IF EXISTS task CASCADE;
```

```
DROP TABLE IF EXISTS project CASCADE;
```

```
DROP TABLE IF EXISTS employee CASCADE;
```

```
DROP TABLE IF EXISTS status CASCADE;
```

-- Таблица сотрудников

```
CREATE TABLE employee (
```

```
    employee_id SERIAL PRIMARY KEY,
```

```
    full_name VARCHAR(100) NOT NULL,
```

```
    email VARCHAR(100) UNIQUE NOT NULL,
```

```
    position VARCHAR(100),
```

```
    department VARCHAR(100),
```

```
    CHECK (email ~* '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$')
```

```
);
```

-- Таблица проектов

```
CREATE TABLE project (
```

```
    project_id SERIAL PRIMARY KEY,
```

```
    name VARCHAR(100) NOT NULL UNIQUE,
```

```
    description TEXT,
```

```
    start_date DATE,
```

```
end_date DATE,  
manager_id INTEGER REFERENCES employee(employee_id) ON DELETE SET NULL,  
CHECK (end_date IS NULL OR end_date >= start_date)  
);
```

-- Таблица статусов

```
CREATE TABLE status (  
    status_id SERIAL PRIMARY KEY,  
    status_name VARCHAR(100) NOT NULL UNIQUE,  
    is_closed BOOLEAN DEFAULT FALSE  
);
```

-- Таблица задач (переименована в task для единообразия)

```
CREATE TABLE task (  
    task_id SERIAL PRIMARY KEY,  
    project_id INTEGER NOT NULL REFERENCES project(project_id) ON DELETE CASCADE,  
    parent_task_id INTEGER REFERENCES task(task_id) ON DELETE CASCADE,  
    status_id INTEGER NOT NULL REFERENCES status(status_id),  
    title VARCHAR(200) NOT NULL,  
    description TEXT,  
    priority VARCHAR(20) DEFAULT 'medium'  
    CHECK (priority IN ('high', 'medium', 'low')),  
    deadline DATE,  
    created_at DATE DEFAULT CURRENT_DATE,  
    created_by INTEGER NOT NULL REFERENCES employee(employee_id)  
);
```

-- Таблица назначений (исправлено название на assignment)

```
CREATE TABLE assignment (  
    assignment_id SERIAL PRIMARY KEY,  
    task_id INTEGER NOT NULL REFERENCES task(task_id) ON DELETE CASCADE,
```

```
employee_id INTEGER NOT NULL REFERENCES employee(employee_id) ON DELETE CASCADE,  
assigned_date DATE DEFAULT CURRENT_DATE  
);
```

-- Таблица комментариев

```
CREATE TABLE comment (  
    comment_id SERIAL PRIMARY KEY,  
    task_id INTEGER NOT NULL REFERENCES task(task_id) ON DELETE CASCADE,  
    employee_id INTEGER NOT NULL REFERENCES employee(employee_id),  
    comment_text TEXT NOT NULL,  
    created_at DATE DEFAULT CURRENT_DATE  
);
```

-- Таблица истории изменений задачи

```
CREATE TABLE taskhistory (  
    taskhistory_id SERIAL PRIMARY KEY,  
    task_id INTEGER NOT NULL REFERENCES task(task_id) ON DELETE CASCADE,  
    changed_by INTEGER NOT NULL REFERENCES employee(employee_id),  
    old_status_id INTEGER REFERENCES status(status_id),  
    new_status_id INTEGER NOT NULL REFERENCES status(status_id),  
    changed_at DATE DEFAULT CURRENT_DATE,  
    changed_why TEXT  
);
```

```
INSERT INTO status (status_name, is_closed) VALUES
```

```
    ('New', FALSE),
```

```
    ('In work', FALSE),
```

```
    ('On check', FALSE),
```

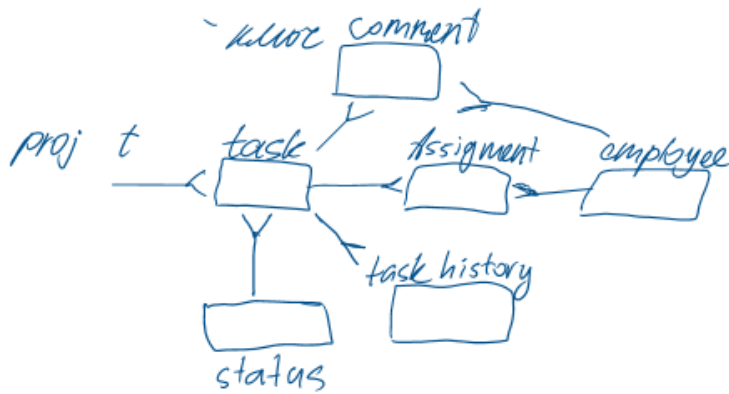
```
    ('Ended', TRUE)
```

```
ON CONFLICT (status_name) DO NOTHING;
```

Бизнес требования:

parent_task_id в Task	Поддержка иерархии задач (декомпозиция крупных задач на подзадачи)
TaskHistory	Требование аудита: необходимо знать, кто и когда менял статус задачи
CHECK (priority IN (...))	Стандартизация приоритетов для корректной фильтрации и отчетов
is_closed в Status	Легко определять завершающие статусы для выборки "активных" задач
ON DELETE CASCADE	При удалении проекта/задачи автоматически удаляются связанные данные (целостность)

gmen -
-id



- loyee task → связь многие ко многим через Assignment
 3ag 2
 1) Pr t
 • pr t_id - первичный ключ
 • n - start date
 • desc ion no date
 • man -id внешний ключ Employee.Employee_id

2) E
 • emp -i - первичный ключ
 • full_
 • email
 • positi
 • dep t
 Stat
 • tat i первичный ключ
 at
 clo